Conception Détaillée EvolutionRobot

Défintions des types et constantes

Booléen CPTR_redresserADroite ← (**digitalRead** (*LINE_SENSOR_RIGH*) = 1) **and**
(**digitalRead** (*LINE_SENSOR_FRONT*) = 0)
Booléen CPTR_redresserAGauche ← (**digitalRead** (*LINE_SENSOR_LEFT*) = 1) **and**
(**digitalRead** (*LINE_SENSOR_FRONT*) = 0)
Booléen CPTR_intersection ← [(**digitalRead** (*LINE_SENSOR_LEFT*) = 1) **and**
(**digitalRead** (*LINE_SENSOR_FRONT*) = 1)] **ou**
[(**digitalRead** (*LINE_SENSOR_RIGHT*) = 1) **and**
(**digitalRead** (*LINE_SENSOR_FRONT*) = 1)]
Enumération Ordre = {AV, TD, TG, . }
Enumération ROBOT_EtatDAvancement = {Avancer, Intersection, Urgence, Sorti}

Procédures

procedure ROBOT_avancer (**E/S :** etat : **ROBOT_EtatDAvancement**)
début

```
tant que (non SENS_instersection) et (etat ≠ Urgence) faire
    si USON_estDevantUnObstacle (ECHO, TRIG) alors
        etat ← Urgence
    sinon
        MTR_avancer (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4,
         PWM_EN1, PWM_EN2)
        si SENS_redresserADroite alors
            MTR_redresser (PWM_EN2, PWM_EN1)
            répéter
                si USON_estDevantUnObstacle (ECHO, TRIG) alors
                    ROBOT_urgence ()
                fin
            jusqu'à digitalRead (LINE_SENSOR_FRONT) = 1;
            MTR_redresser (PWM_EN1, PWM_EN2)
            attendre ()
            MTR_avancer (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4,
             PWM_EN1, PWM_EN2)
        sinon si SENS_redresserAGauche alors
            MTR_redresser (PWM_EN1, PWM_EN2)
            répéter
                si USON_estDevantUnObstacle (ECHO, TRIG) alors
                    ROBOT_urgence ()
                fin
            jusqu'à digitalRead (LINE_SENSOR_FRONT) = 1;
            MTR_redresser (PWM_EN2, PWM_EN1)
            attendre ()
            MTR_avancer (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4,
             PWM_EN1, PWM_EN2)
        fin
        etat ← Intersection
    fin
fin

fin
```

```
procedure ROBOT_intersection (E/S : etat : ROBOT_EtatDAvancement, E : prochaineAction : ORD_Ordre)
début
    suivant prochainVirage faire
        cas où TD faire
            MTR_tournerDroite (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4)
            attendre()
            etat ← Avancer
        cas où TG faire
            MTR_tournerGauche (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4)
            attendre()
            etat ← Avancer
        cas où AV faire
            etat ← Avancer
        autres cas faire
            etat = Sorti
        fin
    fin
fin

procedure ROBOT_urgence ()
début
    tant que USON_ estDevantUnObstacle (ECHO, TRIG) faire
        MTR_arreter (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4)
    fin
    attendre (1000)
fin

procedure ROBOT_evolutionRobot (E : listeDOrdres : ORD_ListeDOrdres)
    Déclaration :  prochaineAction : ORD_Ordre
                   etat : ROBOT_EtatDAvancement
                   numOrdre : Naturel Non Nul
début
    numOrdre ← 1
    etat ← Avancer
    prochaineAction ← listeDOrdre[numOrdre]
    tant que etat ≠ Sorti et prochaineAction ≠. faire
        suivant etat faire
            cas où Avancer faire
                ROBOT_avancer (etat)
            fin
            cas où Intersection faire
                tourner (etat)
            fin
            cas où Urgence faire
                ROBOT_urgence ()
            fin
        fin
        numOrdre ← numOrdre + 1
        prochaineAction ← listeOrdre[numOrdre]
    fin
fin
```